Aquário Plantado Inteligente — Planejamento do Trabalho

- **Disciplina:** Distribuição de Processos e Dados (UFC), Prof. Dr. Paulo A. L. Rego
- **Entregáveis:** slides, código-fonte (este repositório), vídeo de até 7 min, nomes e matrículas no formulário
- **Versão:** 2 (26/10/2026). Acrescenta a estrutura do repositório, integração, execução, testes e gravação.

1. Motivação

O que é um aquário plantado

É um aquário em que as **plantas vivas são as protagonistas**, não só os peixes. Funciona como um jardim debaixo d'água. Para as plantas crescerem, o aquarista precisa controlar o mesmo que uma planta terrestre precisa: **luz, carbono (CO2) e nutrientes**. Quem manda no resultado é o equilíbrio entre os três:

- **Luz demais ou CO2 de menos:** as plantas não conseguem usar a energia e as **algas tomam conta** do aquário.
- **CO2 demais:** os peixes **sufocam**. É a causa mais comum de perder todos os peixes de uma vez nesse hobby.
- **Temperatura errada:** os peixes tropicais adoecem, e a água quente segura menos oxigênio.

O problema

Hoje, o aquarista típico automatiza cada equipamento **de forma isolada**: um timer para a luz, outro para o CO2, cada um no seu app e sem conversar entre si. Isso significa:

- **Ninguém está olhando:** se o CO2 trava ligado, o aquecedor queima ou a lâmpada para de funcionar, só se descobre quando os peixes já morreram ou as algas apareceram.
- **Não há coordenação:** o CO2 deveria seguir a luz (as plantas só consomem CO2 durante a fotossíntese), mas os timers nem sabem que o outro existe.
- **Não há histórico:** é impossível responder perguntas como "quanto tempo de luz este aquário recebeu esta semana?" ou "o pH caiu demais ontem?", que é justamente o que o aquarista precisa para ajustar o equilíbrio.

A solução

Um **sistema central** que **enxerga todos os equipamentos**, **guarda o histórico**, **avisa na hora** quando algo sai do normal e permite **comandar tudo de um só lugar**. É exatamente a arquitetura pedida no trabalho (Gateway, fontes de dados e Cliente), aplicada a um problema real, com equipamentos reais.

Por que cada equipamento importa

Equipamento	Por que é importante
Luminária principal (Smartled)	É o "sol" das plantas. A **intensidade** e a **duração** (fotoperíodo, normalmente 6–8 h) definem o crescimento das plantas, ou das algas. A rampa de amanhecer/entardecer evita o susto dos peixes com a luz acendendo de uma vez.
Luminária secundária	Complementa a luz principal. Pode ter outro espectro de cor ou ficar ligada por menos tempo, para dar um pico de luz no meio do dia.
CO2	É o "adubo" principal das plantas, e também **o equipamento mais perigoso**: em excesso, mata os peixes. Deve ligar antes da luz e desligar antes de ela apagar. Por isso o sistema pode **desligá-lo automaticamente** quando algo dá errado.
Sensor de pH	O CO2 dissolvido deixa a água mais ácida, então o pH funciona como um **termômetro do CO2**. Se o pH cai demais, o sistema dispara um alerta imediato: é o aviso de que os peixes estão em risco.
Sensor de temperatura	Os peixes e as plantas tropicais vivem entre ~24 e 28 °C. Detecta falha do aquecedor ou superaquecimento no verão. Água quente com CO2 alto é a combinação mais perigosa.
Medição de consumo (nas tomadas)	Confirma que o equipamento **realmente está funcionando**, e não só que a tomada está ligada. Luz ligada com 0 W significa lâmpada queimada; CO2 ligado com 0 W significa válvula com defeito. Também permite calcular o gasto de energia.

> *Um aquário plantado é um ecossistema frágil em que **luz, CO2 e temperatura precisam andar juntos**. Este sistema distribuído conecta os equipamentos reais do aquário para **monitorar, coordenar e alertar**, e assim evitar algas e, principalmente, a perda dos peixes.*

Os números (6–8 h, 24–28 °C, pH 6,4) são as faixas usuais do hobby. O limite de pH depende do pH da água antes de entrar o CO2 e deve ser ajustado para cada aquário.

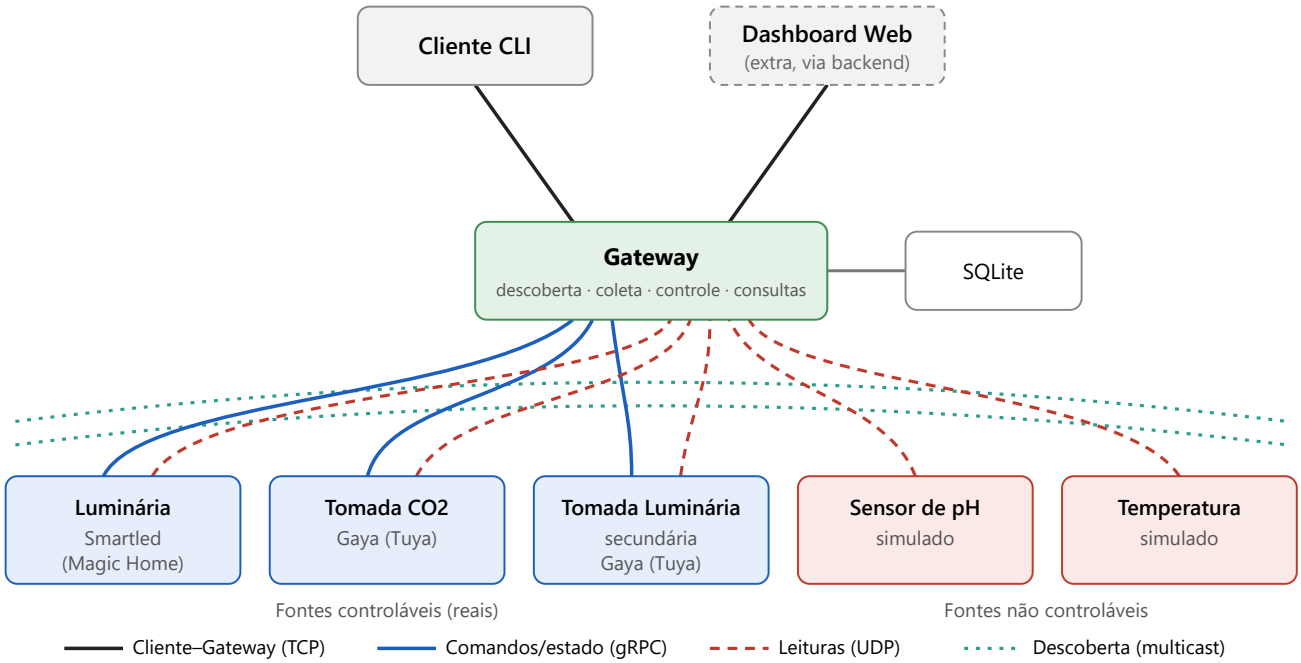
2. Hardware real disponível

Equipamento	Dispositivo	Protocolo por baixo	Biblioteca	Status
Luminária principal	Smartled, controlador Magic Home "Controller Dimmable (0x41)"	TCP 5577, sem chave	`flux_led`	✅ leitura e comando testados
CO2	Tomada Wi-Fi Gaya 9812 (Tuya)	Tuya v3.3, TCP 6668, criptografado (local key)	`tinytuya`	✅ leitura testada
Luminária secundária	Tomada Wi-Fi Gaya 9812 (Tuya)	Tuya v3.3, TCP 6668, criptografado (local key)	`tinytuya`	✅ leitura testada
pH / Temperatura	Simulados	—	—	a implementar

As tomadas **medem energia**: potência (W), corrente (mA) e tensão (V).

Argumento central para a apresentação: o Gateway **não sabe nada de Tuya nem de Magic Home**. Ele só fala gRPC, UDP e Protobuf. Cada adaptador esconde o protocolo proprietário, então trocar um equipamento real por um simulado, ou por outra marca, não muda nada no Gateway nem no Cliente.

3. Arquitetura



Todas as mensagens são serializadas com Protocol Buffers. O contrato completo e as portas estão em `proto/README.md`.

Fluxo	Protocolo	Endereço
Descoberta: pedido (Gateway → fontes), ao iniciar e a cada 30 s	UDP multicast	`239.10.10.10:50000`
Descoberta: resposta (fonte → Gateway)	UDP unicast	`gateway:50001`
Leituras (fontes → Gateway)	UDP unicast	`gateway:50002`
Comandos (Gateway → fontes controláveis)	gRPC	`:50051` luminária, `:50052` CO2, `:50053` tomada luminária
Cliente ↔ Gateway	TCP, prefixo de 4 bytes	`gateway:50010`
Dashboard (extra)	HTTP (Next.js)	`:3000`

4. Componentes

Processo	Papel	gRPC?	Dados enviados por UDP	Evento imediato
Luminária Smartled	liga/desliga, intensidade %, rampa de amanhecer/entardecer, frequência de envio	✅	ligada, intensidade	mudança de estado
Tomada CO2	liga/desliga, frequência de envio; **failsafe**: desliga se perder o Gateway por 90 s	✅	ligada, W, mA, V	mudança de estado, ligada sem consumo
Tomada luminária secundária	liga/desliga, frequência de envio	✅	ligada, W, mA, V	mudança de estado, ligada sem consumo
Sensor de pH	simulado: cai com o CO2 ligado e sobe quando desliga	❌	pH	**pH < 6,4**
Sensor de temperatura	simulado: oscila perto de 26 °C	❌	°C	**> 30 °C**
Gateway	descoberta, recepção UDP, comandos gRPC, servidor TCP, consultas, SQLite, detecção de offline, **desliga o CO2 com pH baixo**	cliente	—	—
Cliente CLI	listar fontes, comandos, consultas, histórico	—	—	—
Dashboard Next.js (extra)	cartões das fontes, gráficos, comandos; conecta ao Gateway por TCP pelos Route Handlers	—	—	—

Os horários das tomadas continuam no app Smart Life. O sistema monitora as mudanças, comanda manualmente e reage a alertas.

5. Consultas agregadas (o requisito pede ≥ 3; teremos 5)

1. **Média e desvio padrão** de uma medida (ex.: temperatura na última hora)
2. **Consumo em Wh** por tomada (dado real de energia)
3. **Tempo ligado** (horas de luz e de CO2)
4. **pH mínimo e número de alertas** de pH
5. **Fonte com maior variação** (coeficiente de variação)

Mais o **histórico** (série temporal) para o CLI e o dashboard. As fórmulas estão em `gateway/README.md`.

6. Pontuação extra

- **Mais de uma linguagem:** Gateway em **Java**; fontes e CLI em **Python**; dashboard em **TypeScript (Next.js)**.
- **Persistência:** SQLite no Gateway, com consulta de histórico pelo Cliente.
- **Interface gráfica:** dashboard web com séries temporais.

7. Estrutura do repositório e responsáveis

Cada pasta tem um `README.md` que detalha **o que implementar**, **como testar** e **quando está pronto**.

Pasta	Conteúdo	Responsável
`proto/`	contrato `aquario.proto` + portas e nomes	equipe
`gateway/`	Gateway (Java + Maven + SQLite)	**A** (consultas: **C**)
`fontes/comum/`	descoberta, envio UDP e laço periódico compartilhados	**B**
`fontes/luminaria/`	Smartled (Magic Home)	**B**
`fontes/tomada/`	tomadas Tuya (CO2 e luminária secundária)	**B**
`fontes/sensor_ph/`, `fontes/sensor_temperatura/`	sensores simulados	**C**
`cliente_cli/`	cliente de linha de comando	**C**
`dashboard/`	Next.js: Route Handlers (cliente TCP do Gateway) + página com gráficos	**D**
`scripts/`	gerar o proto, subir e parar o sistema	**A** (integração)
`apresentacao/`	slides, roteiro e gravação do vídeo	**D** (com todos)

Fluxo de trabalho no Git

- Um branch por pessoa: `gateway`, `fontes-reais`, `simulados-cli`, `dashboard`. PR para o `main` com revisão de outra pessoa.
- **Mudança no `.proto` só por PR próprio**, avisando o grupo, porque quebra todo mundo.
- Ninguém sobe `fontes/config.json` (tem as chaves das tomadas).

8. Integração

A integração é feita **por marcos**. Cada marco só começa quando o anterior funciona **com o backend `sim`**.

Marco	O que precisa funcionar	Envolvidos	Como verificar
M0 — Contrato	`aquario.proto` gera código em Python e Java sem erro	todos	`protoc` e `mvn package` passam
M1 — Descoberta	o Gateway lista as 5 fontes simuladas	A, B, C	log `[FONTE] ... ONLINE` × 5
M2 — Dados	leituras chegando e gravadas no SQLite	A, B, C	`fontes` no CLI mostra a última leitura
M3 — Comandos	CLI → Gateway → gRPC → fonte → Status de volta	A, B, C	`ligar tomada-co2` muda o estado
M4 — Consultas	as 5 consultas e o histórico respondem corretamente	A, C	conferir com valores esperados (seção 10)
M5 — Falhas	offline/online, Gateway reiniciando, failsafe do CO2	todos	cenários F1–F6 (seção 10)
M6 — Hardware real	backends `magichome` e `tuya` no lugar do `sim`	B	luminária e tomadas respondendo
M7 — Dashboard	dashboard sobre o sistema completo	D	gráficos atualizando

Ponto de encontro: fazer a integração **na máquina que tem acesso ao aquário** (rede `192.168.0.x`). Os marcos M0–M5 podem ser feitos em qualquer PC com o backend `sim`.

9. Execução

Preparação (uma vez por máquina)

```
python -m pip install -r requirements.txt
python -m grpc_tools.protoc -I proto --python_out=gen/python --grpc_python_out=gen/python
proto/aquario.proto
cd gateway && mvn -q package
cd ../dashboard && npm install
```

Para o hardware real: copiar `fontes/config.exemplo.json` para `fontes/config.json` e preencher (as keys saem do `python -m tinytuya wizard`).

Checklist antes de rodar

- [ ] VPN desligada; IP da rede local anotado (`ipconfig`)
- [ ] Firewall do Windows liberando Python e Java em rede privada
- [ ] Tomada do CO2 configurada para voltar **desligada** depois de queda de energia (DPS 38, pelo app)

Subida

```
./scripts/subir_tudo.ps1 -IpLocal 192.168.0.33 -Backend sim    # desenvolvimento e ensaio
./scripts/subir_tudo.ps1 -IpLocal 192.168.0.33 -Backend real   # demo com o aquário
```

Ordem: Gateway → tomadas → luminária → sensor de pH → temperatura → CLI → dashboard (detalhes em `scripts/README.md`).

10. Testes

10.1 Testes unitários (cada responsável, na própria pasta)

Componente	O que testar
Gateway — consultas	Dados fixos no SQLite em memória: 2 leituras de 10 W com 1 h de diferença → **10 Wh**; média e desvio padrão populacional de [24, 26, 28] → **26 / 1,63**; tempo ligado ignorando lacunas > 3 × intervalo
Gateway — enquadramento TCP	Mensagem partida em 2 pedaços é lida inteira
Sensor de pH	De 7,2 com alvo 6,2, depois de 3·tau → ≈ 6,25; evento só ao cruzar o limiar (com histerese)
Tomada	Conversão DPS → W/mA/V; evento `ligada_sem_consumo` depois de 2 leituras
CLI	Parser de comandos: `param luminaria-smartled intensidade 40` → `Comando` correto

10.2 Testes de integração (cenários com backend `sim`)

#	Cenário	Resultado esperado
I1	Subir o Gateway e depois as 5 fontes	Todas aparecem ONLINE em ≤ 30 s
I2	Subir as fontes **antes** do Gateway	Aparecem assim que o Gateway sobe
I3	`param sensor...` em fonte não controlável	Erro claro: "fonte não controlável"
I4	`param tomada-co2 intervalo_envio_s 3`	Leituras passam a chegar a cada 3 s
I5	CO2 ligado + pH com `--ph-com-co2 6.2`	Evento `ph_baixo` → Gateway desliga o CO2 → pH volta a subir
I6	`temperatura --cenario superaquecimento`	Evento `temperatura_alta` no Gateway
I7	As 5 consultas no CLI	Valores coerentes com o que foi enviado

10.3 Cenários de falha (exigidos no vídeo)

#	Falha	Resultado esperado
F1	Matar uma fonte (Ctrl+C)	Gateway marca OFFLINE em ≤ 3 intervalos
F2	Reiniciar a fonte	Volta a ONLINE sozinha
F3	`falha tomada-luminaria on`	Para de enviar → OFFLINE; `off` → volta
F4	Comando para fonte offline	Erro no CLI (timeout gRPC), sem travar nada
F5	Reiniciar o Gateway	Redescobre todas as fontes; CLI reconecta
F6	Matar o Gateway com o CO2 ligado	Em ~90 s a tomada do CO2 desliga sozinha (failsafe)

10.4 Checklist dos requisitos da especificação

Requisito	Onde é atendido	Verificado em
1a — Protobuf em todas as mensagens	`proto/aquario.proto`	M0
1b-i — TCP Cliente↔Gateway	`cliente_cli/`, `gateway/`	M3
1b-ii — gRPC Gateway→fontes controláveis	serviço `FonteControlavel`	M3
1b-iii — UDP para os dados	`Leitura` → :50002	M2
1b-iv — multicast na descoberta	`DiscoveryRequest` → 239.10.10.10	M1
2a — cada fonte é um processo	`fontes/*`	I1
2b — fontes informam tipo, IP/porta e estado inicial	`DiscoveryResponse`	M1
3 — cliente lista, comanda e consulta	`cliente_cli/`	M3–M4
4 — Gateway pede identificação ao iniciar	descoberta	I1, I2
5b — ≥ 3 tipos, ≥ 1 controlável, ≥ 1 não controlável	5 fontes (3 controláveis, 2 não)	I1
5c — envio imediato por evento	pH, temperatura, tomadas	I5, I6
5d — ≥ 3 consultas agregadas	5 consultas	I7
Extras — linguagens, persistência, GUI	Java + Python + TypeScript, SQLite, dashboard Next.js	M4, M7

11. Gravação e entrega

Instruções completas em `apresentacao/README.md`. Resumo:

1. **Ensaio** com o backend `sim` seguindo o roteiro e cronometrando (meta: 6 min 30 s).
2. **Gravação** com o backend `real`: OBS gravando a tela (terminais com título + dashboard) e, se possível, uma câmera (celular) mostrando o aquário, para a luminária mudando de brilho de verdade.
3. **Upload** no YouTube como **não listado**; conferir que o link abre em aba anônima.
4. **Formulário:** nomes e matrículas, slides (PDF), link do repositório, link do vídeo.

12. Cronograma

O prazo original (25/10/2026) já passou. O cronograma está em **dias relativos**: preencha a coluna "Data" assim que o grupo confirmar a nova data com o professor.

Dia	Data	Entrega
Dia 1		Repositório clonado por todos; **M0** (proto gerando em Python e Java); branches criados
Dia 2		`fontes/comum` + Gateway com descoberta → **M1**; sensores simulados enviando → **M2**
Dia 3		Comandos gRPC e CLI → **M3**; consultas → **M4**
Dia 4		Cenários de falha → **M5**; `scripts/subir_tudo.ps1`; dashboard em paralelo
Dia 5		Hardware real → **M6**; dashboard integrado → **M7**; slides prontos
Dia 6		Ensaio cronometrado + **gravação** + upload
Dia 7		Revisão final e **envio do formulário** (margem)

13. Riscos e cuidados

Risco	Mitigação
Prazo	Confirmar a nova data com o professor; priorizar M0–M5 (obrigatórios) antes dos extras
Grupo de 4 pessoas	A especificação permite até 3. Confirmar com o professor
CO2 ligado sem supervisão	Failsafe na fonte (90 s sem Gateway), desligamento automático com pH baixo, DPS 38 = desligada, testes reais curtos e supervisionados
VPN ou várias interfaces de rede	`--ip-local` obrigatório em todos os processos; VPN desligada na demo
IPs mudando (DHCP)	Reserva no roteador; IPs só no `config.json`
Chaves no repositório público	`config.json`, `devices.json`, `tinytuya.json` no `.gitignore`; revisar o `git status` antes de cada commit
Hardware falhar no dia do vídeo	Todos os adaptadores com `--backend sim`; gravar a versão simulada se precisar
Mudança no `.proto` quebrando os outros	Só por PR próprio, com aviso ao grupo